

BEFORE Placeholder

AFTER

# Integration Pattern: SaaS / API Integration (Read & Write)

Document classification: Internal

Pattern owner: Security Engineering/Architecture

Version: 0.1

Approved for use with tiers: ☐ Critical ☐ Important ☐ Low (case-by-case)

Last review: \_\_\_\_\_

## 1. Purpose

This pattern defines a secure-by-design approach for integrating third-party SaaS/services via API, including least-privilege scopes, credential handling, logging/auditability, and change control.

## 2. When to use

Use this pattern when:

- a supplier requires API access to organisational systems/data, or
- internal systems call supplier APIs to read/write data, or
- an automation/integration platform (iPaaS) is used to connect services.

## 3. Default position (principles)

- Prefer **read-only** integrations unless write access is essential.
- Prefer **short-lived tokens** and centrally managed secrets over long-lived keys.
- Use **separate identities** per environment (prod vs non-prod) and per integration purpose.

- Enforce **least privilege** scopes and **explicit allowlists** (endpoints, data objects, IPs) where supported.
- Ensure **auditability**: admin/auth events and integration actions must be logged and retrievable.

## 4. Pattern variants

### Variant A — Read-only API integration (preferred)

Applies to: reporting, data pulls, enrichment, monitoring.

Minimum controls:

- Read-only scopes/roles
- Rate limiting and pagination to avoid bulk extraction where feasible
- Restrict accessible datasets/objects to the minimum required

### Variant B — Write API integration (higher risk)

Applies to: provisioning, account updates, workflow automation, changes to critical systems.

Additional controls required:

- Separate write identity (do not reuse read identity)
- Approval/change control for scope changes
- Stronger monitoring and alerting for write actions

## 5. Minimum requirements (baseline)

### 5.1 Identity model

MUST:

- Use a dedicated **service account / app registration** for the integration (no personal accounts).
- Separate identities for:

- production vs non-production
- read vs write (where write is required)
- Restrict who can administer the integration identity (admin RBAC).

SHOULD:

- Where supported, use workload identity / client credentials flow with managed identity equivalent.
- Enforce "break-glass" only for integration admin functions, not daily operations.

## 5.2 Authentication and credential handling

MUST:

- Store tokens/keys/secrets in an approved secrets store (vault/secure store); never in code or plaintext configs.
- Implement rotation:
  - on role change for admins,
  - on suspected compromise,
  - on schedule (define schedule per tier).
- Disable/remove credentials immediately when integration is decommissioned.

SHOULD:

- Prefer short-lived tokens over static API keys.
- Prefer mTLS where supported for service-to-service integrations.

## 5.3 Authorization (least privilege)

MUST:

- Document required API endpoints/objects and map them to scopes/permissions.
- Grant only the minimum scopes required for the defined use case.
- For write integrations, implement additional controls to constrain blast radius (object-level permissions, scoped roles, approval gates where possible).

SHOULD:

- Implement scope “deny by default” posture with explicit allowlists.
- Use separate tenant/app instances if the supplier supports it for segmentation.

## 5.4 Network controls (where applicable)

MUST (if supported/required):

- Restrict integration calls via allowlisted egress IPs, private endpoints, or approved connectivity patterns.
- Block inbound callbacks unless explicitly required and approved.

SHOULD:

- Prefer private connectivity options for Critical integrations when available and justified.

## 5.5 Logging and auditability

MUST:

- Enable audit logs for:
  - integration identity authentication events
  - token issuance/refresh where available
  - admin changes to integration configuration/scopes
  - API actions relevant to the integration (especially write actions)
- Define and document log retention expectations and retrieval method (pull/export/push).

SHOULD:

- Create alerting for:
  - unusual volume of API calls
  - new geolocations/IPs (where visible)
  - permission/scope changes
  - write actions outside expected schedules

## 5.6 Change control

MUST:

- Treat scope/permission changes as a controlled change (ticketed, reviewed, and approved).
- Re-run tiering/triage if integration changes materially (read→write, new systems, new data types).

SHOULD:

- Use versioned configuration (infrastructure as code where applicable) for integration settings.

## 6. Implementation steps (high level)

1. Confirm tier and whether write access is required (challenge necessity).
2. Document integration data flows (source → destination, data types, frequency).
3. Create dedicated integration identity (separate prod/non-prod; read/write).
4. Apply least privilege scopes and restrict admin access.
5. Store secrets in vault; configure rotation and ownership.
6. Enable audit logging; confirm retention and retrieval.
7. Validate with test transactions (read and/or controlled write).
8. Record evidence and sign off prior to go-live.

## 7. Validation checklist (sign-off)

Record evidence location for each item.

Identity & access: ☐ Dedicated integration identity created (no personal accounts)

☐ Prod/non-prod separation confirmed

☐ Read/write separation confirmed (if write)

☐ Admin RBAC restrictions applied

Secrets: ☐ Secrets stored in approved vault/store

☐ Rotation process defined + owner assigned

☐ Emergency revoke steps documented

Permissions: ☐ Scopes mapped to endpoints/objects

☐ Least privilege confirmed by review

☐ Write controls documented (if applicable)

Logging: ☐ Auth/audit logs enabled

☐ Admin changes logged

☐ Integration actions logged (esp. writes)

☐ Retention/retrieval confirmed

Network (if applicable): ☐ Egress allowlist/private connectivity configured

☐ Inbound callbacks avoided or approved

Change control: ☐ Scope change process documented and enforced

Sign-off: Security Engineering/Architecture (role title): \_\_\_\_\_ Date: \_\_\_\_\_ IT/Platform

Owner (role title): \_\_\_\_\_ Date: \_\_\_\_\_ Business Service Owner (role title):

\_\_\_\_\_ Date: \_\_\_\_\_

## 8. Offboarding notes (API integration teardown)

On termination/disablement:

- revoke/rotate tokens and secrets immediately
- disable integration identity
- remove allowlists/private endpoints and routes
- retain relevant audit logs per retention policy
- confirm no scheduled jobs remain active